CSE_3523

Segment-6
I/O operation

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Introduction to I/O Interface

The I/O interface allows communication between the microprocessor and external devices . ***IN*** and **OUT Instruction** transfer data between an I/O device and the microprocessor's accumulator (AL, AX or EAX).

IN/OUT Operation Examples Based on Data and Address Size

#	Address Type	Data Size	Example Instruction	Meaning
1	8-bit fixed	8-bit	IN AL, 60H OUT 60H, AL	I/O with fixed port 60H using AL
2	16-bit via DX	8-bit	MOV DX, 03F8H IN AL, DX OUT DX, AL	I/O with port in DX using AL
3	16-bit via DX	16-bit	MOV DX, 0378H IN AX, DX OUT DX, AX	I/O with port in DX using AX
4	8-bit fixed	16-bit	IN AX, 19H OUT 19H, AX	I/O with fixed port 0019H using AX

Addressing Techniques for I/O Instructions

There are two ways to specify the address of an I/O port:

- **Fixed I/O Addressing (8-bit)**
  - The port address is within the instruction.(e.g. IN AL, 60H)
  - It appears on address lines A7–A0, while A15–A8 are 00000000b.
  - Used for accessing **first 256 ports** (00H–FFH).
  - used to access system board device,(e.g. keyboard.)
- **Variable I/O Addressing (16-bit)**
  - The port address is stored in the **DX register**.
  - The full 16-bit address appears on address lines A15–A0.
  - Enables access to **full 64K I/O space**.
  - used to access serial and parallel ports, harddrives, etc.

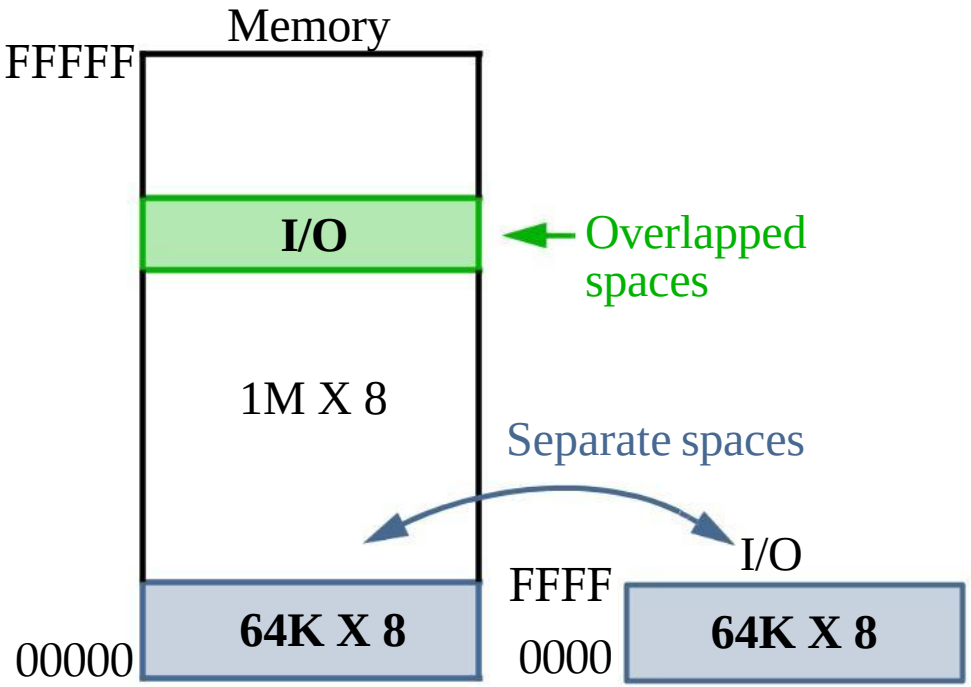
```
MOV DX, 0378H
IN AX, DX
```

Isolated Vs Memory Mapped

Isolated I/O

- Uses **separate address space** for I/O.
- Requires **special instructions**: IN, OUT.
- Hardware using M/IO andW/R needed to develop signals IORC
- and IOWC.

$\overline{RD}$	$\overline{WR}$	$IO/\overline{M}$	Signal
0	1	0	$MEMR$
1	0	0	ME**MW**
0	1	1	$\overline{IOR}$
1	**0**	**1**	$\overline{IOW}$



Memory-Mapped I/O

- I/O shares **memory address space**.
- Uses **normal instructions**: MOV, ADD, etc.

IORC and IOWC not required.

- 8-bit port addresses used to access system board device, e.g. timer and keyboard.

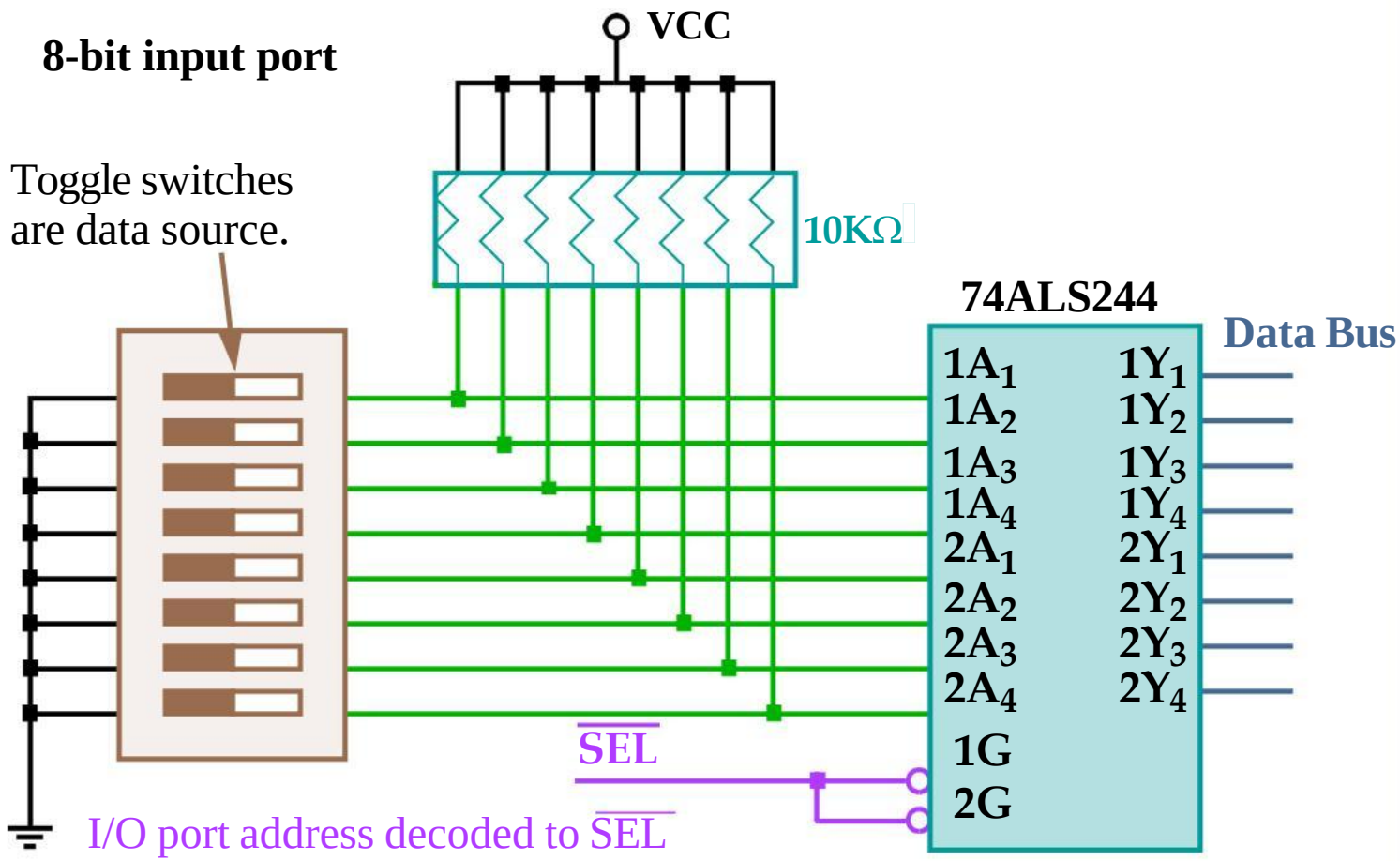
Basic I/O Interface

Basic Input Interface: The basic input device (to the microprocessor) is a set of tri-state buffers(**74ALS244**)

The **74ALS244 tri-state buffer** reads input from **toggle switches** and places 8-bit data on the CPU data bus.

A **decoded SEL signal** enables the buffer to transfer data during an IN instruction.

10kΩ pull-up resistors ensure proper logic levels for stable input.



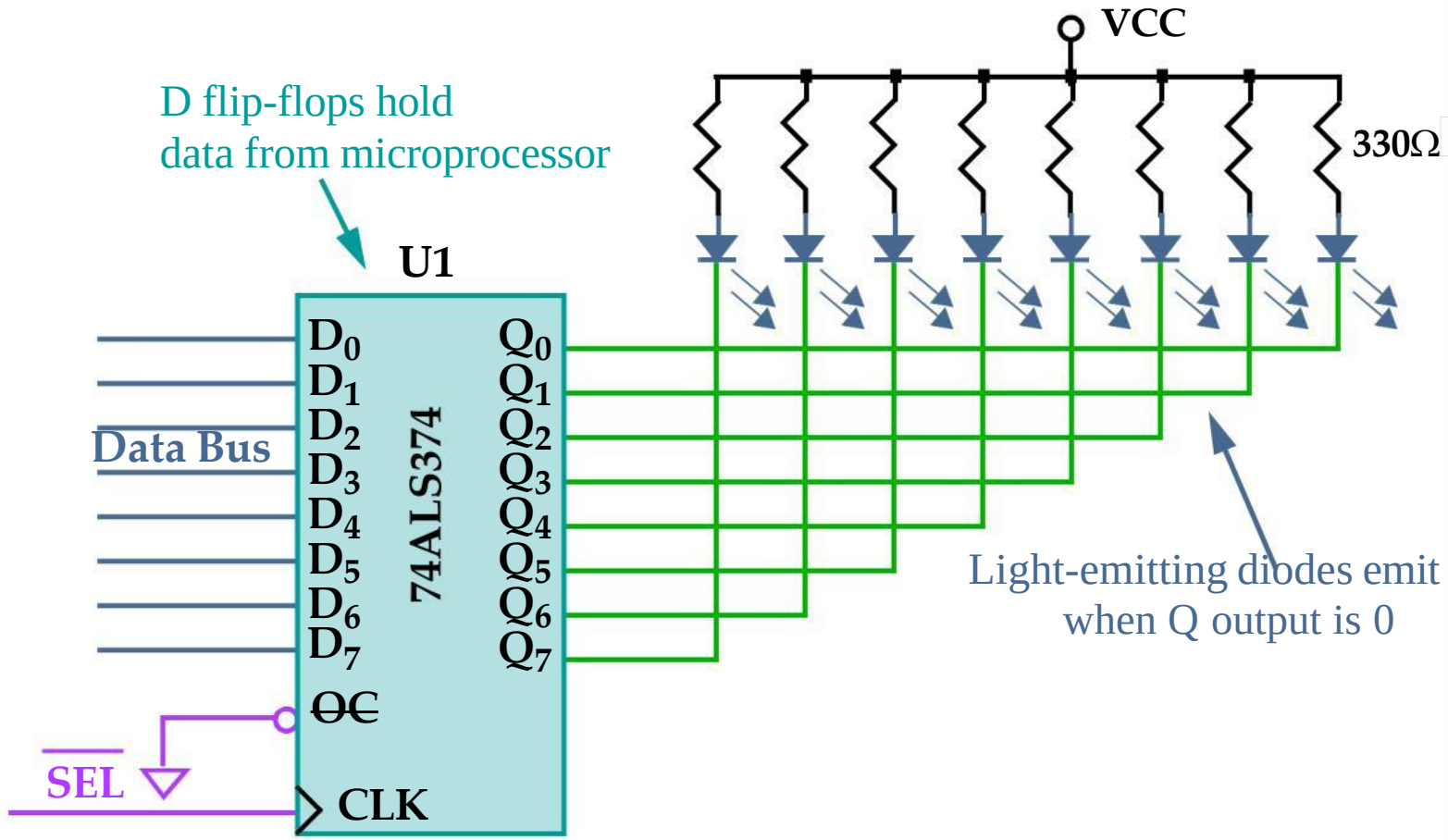
Basic I/O Interface

Basic Output Interface: The basic output device (from the microprocessor) is a set of latches.

ca ptures output data from the CPU using the OUT instruction.

A **SEL signal** triggers the latch to store data, which controls **LEDs** connected to its outputs.

330Ω resistors limit current, and LEDs turn ON when output is logic 0



Minimum Mode Memory Interfacing

In minimum mode, the **address bus and data bus are multiplexed** on the same lines (AD0 to AD15).

- During the first clock cycle, these lines act as a **memory/IO address bus**.
- For the second and third clock cycles, they function as a **data bus**, carrying data.
- **Demultiplexing** is the process of separating these address and data signals for read/write operations.

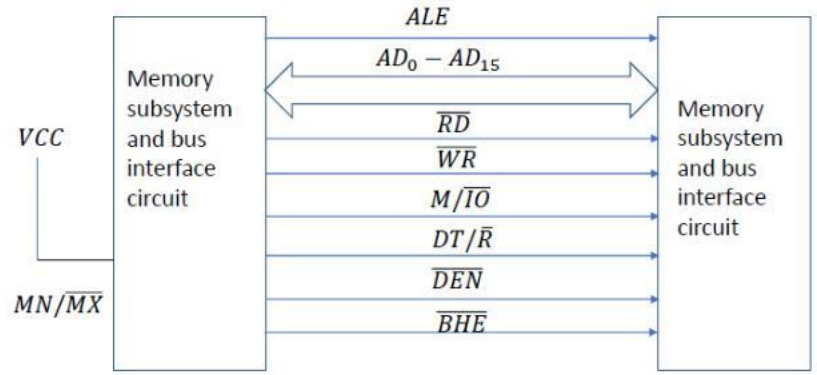


Figure (2) block diagram for minimum mode 8086 memory interface

The control signals in minimum mode are listed in table below

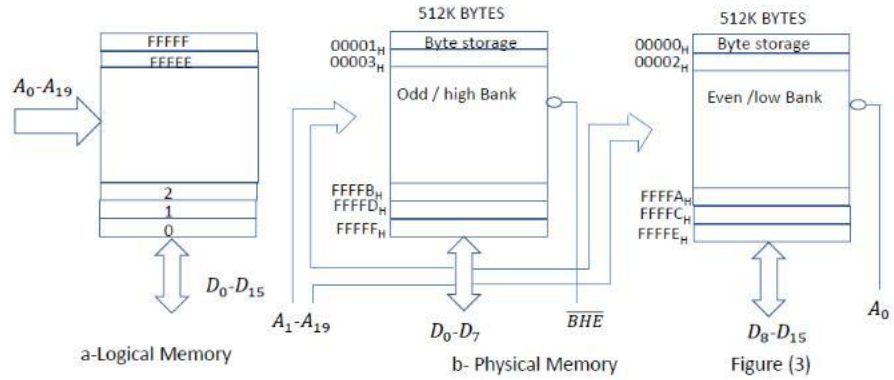
$\overline{RD}$	$\overline{WR}$	$IO/\overline{M}$	Signal
0	1	0	$\overline{MEMR}$
1	0	0	$\overline{MEMW}$
0	1	1	$\overline{IOR}$
1	0	1	$\overline{IOW}$

8086 Memory Organization

The 8086 microprocessor's memory address space has distinct logical and physical organizations.

- **Logical Organization**: The memory is seen as a **single 1M×8 memory bank**. Byte-wide storage locations are assigned consecutive addresses from 00000H to FFFFFH.

- **Physical Organization**: Physically, memory is implemented as **two independent 512 Kbyte banks**: the **low (even) bank** and the **high (odd) bank**



$\overline{BHE}$	A_0	Function
0	0	Choose both odd and even memory bank
0	1	Choose only odd memory bank
1	0	Choose only even memory bank
1	1	None is chosen

Address Decoding

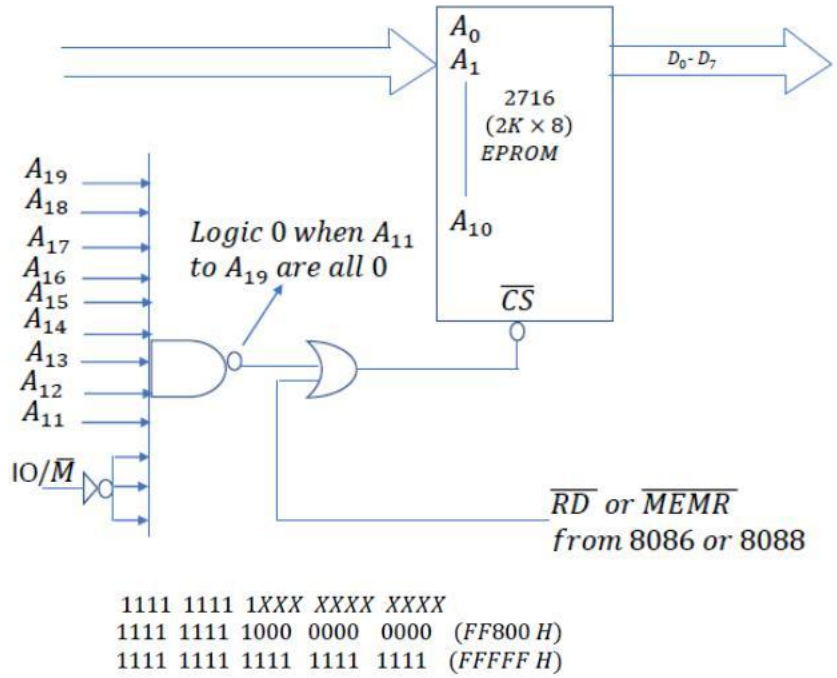
A microprocessor can typically address a much larger memory space than a single memory chip can cover. This is where **address decoding** becomes essential.

- **Purpose of Decoding**: Decoding ensures that a memory device functions at a **unique section or partition of the memory map**.
- **Without a Decoder**: Only one memory device could be connected to a microprocessor, severely limiting its utility.
- **Correcting Mismatches**: A decoder corrects mismatches when the microprocessor's address pins exceed the memory component's address pins. For instance, the 8088 has 20 address connections, while a 2716 EPROM has only 11.

NAND Gate Decoder

a 2K × 8 EPROM, like the 2716, being used with an 8088 microprocessor.

- The 8088's address connections A10–A0 are directly connected to the EPROM's address inputs A10–A0.
- The remaining nine address pins (A19–A11) are connected to a **NAND gate decoder**.

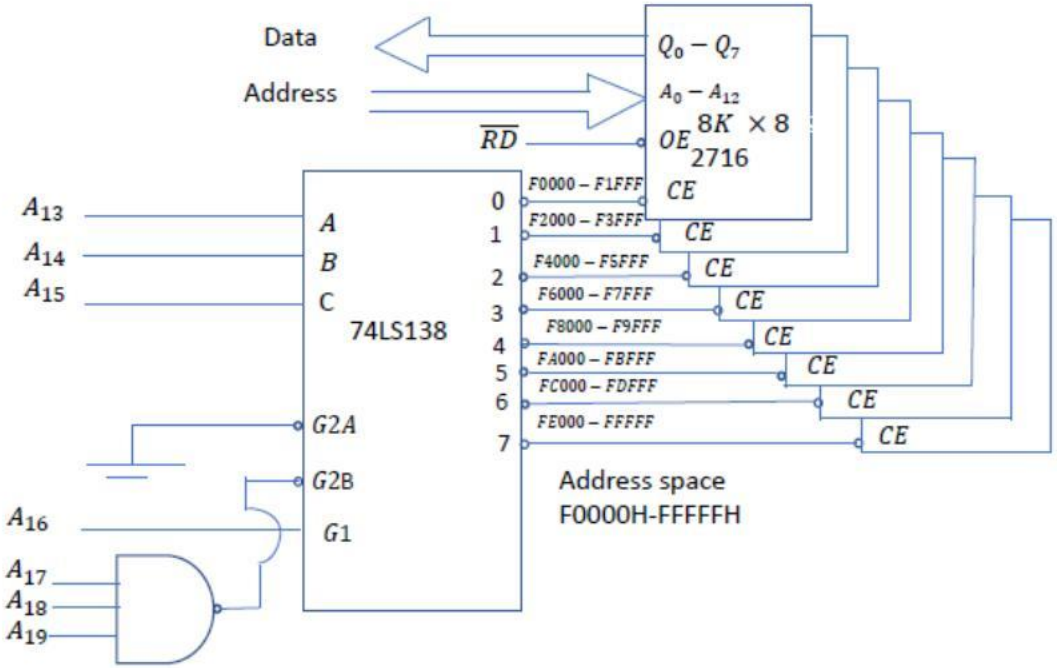


- This decoder selects the EPROM from one of the 2K-byte sections within the 8088's 1M-byte memory system.

A simple NAND gate decoder can be used to map a memory device to a specific address range, such as FF800H to FFFFFH

Using a 3-to-8 Decoder (74LS138)

For more complex memory systems, a **3-to-8 decoder** like the 74LS138 is often used.In a system with eight 2764 EPROMs a 64K × 8 memory section in an 8088 microprocessor:

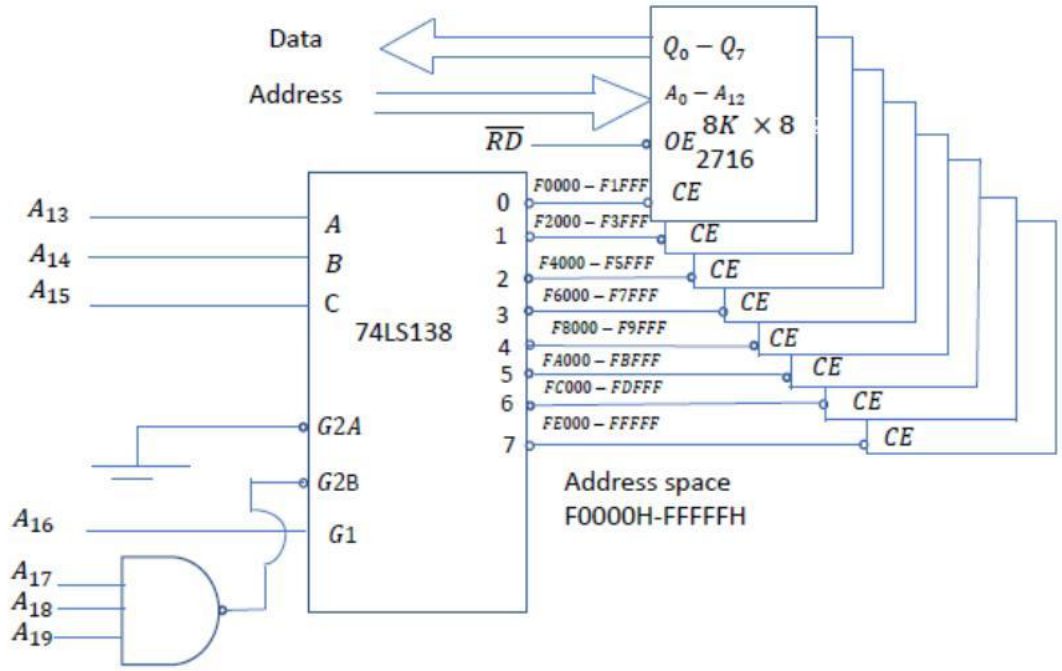


A circuit that uses eight 2764 EPROMs for a 64K × 8 section of memory in an 8088 microprocessor-based system. The addresses selected in this circuit are F0000H–FFFFFH.

- Address bits A19–A17 are connected to a three-input NAND gate. When all three are high, the NAND gate output goes low, enabling input G2B of the 74LS138.
- Input G1 is directly connected to A16.
- For this decoder to be enabled, the first four address connections (A19–A16) must all be high

Using a 3-to-8 Decoder (74LS138)

- Address inputs C, B, and A of the 74LS138 connect to microprocessor address pins A15–A13. These inputs determine which of the decoder's output pins goes low, selecting a specific EPROM.



Module 0
1111 000X XXXX XXXX XXXX
1111 0000 0000 0000 0000 =F0000H
1111 0001 1111 1111 1111 =F1FFFH
Module 7
1111 111X XXXX XXXX XXXX
1111 1110 0000 0000 0000 =FE000H
1111 1111 1111 1111 1111 =FFFFFH

Memory Expansion

There are two primary reasons to expand memory capacity beyond a single device:

1. The **byte-wide length is insufficient**.
2. The **total storage capacity isn't enough bytes**.

Both needs can be met by interconnecting multiple Integrated Circuits (ICs).

Memory Expansion

Example 1: Implementing 32Kx16 EPROM using 32Kx8 EPROMs

To implement a 32K × 16 EPROM using 32K × 8 EPROMs, you'd use two 32K × 8 chips in parallel.

- **A 32K × 8 EPROM** has 15 address lines (A0–A14) (2^{15}=32K).
- **A 32K × 16 EPROM** also needs 15 address lines (A0–A14).
- One 32K × 8 EPROM handles data lines D0–D7, and the other handles D8–D15 to achieve the 16-bit wide data path.

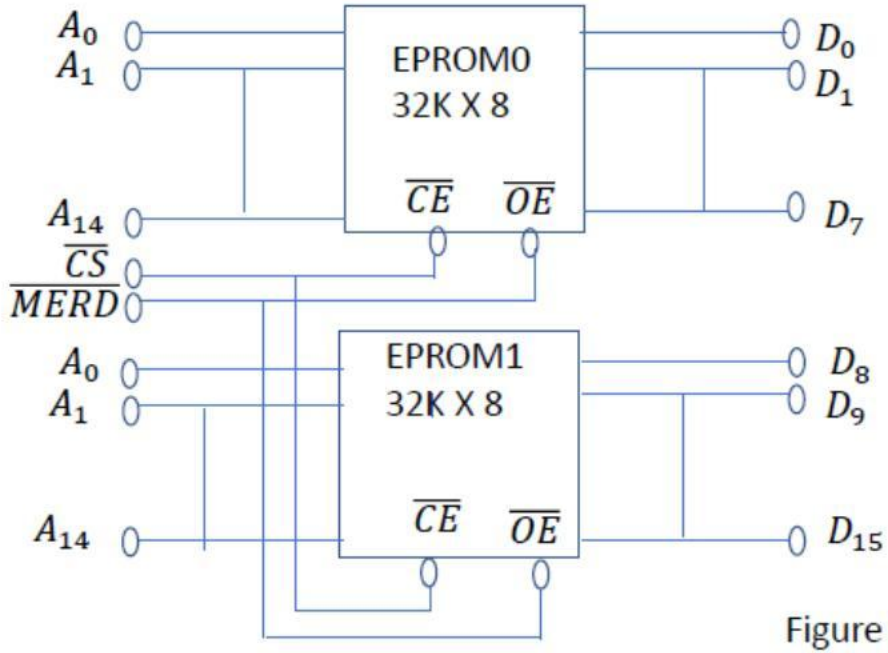


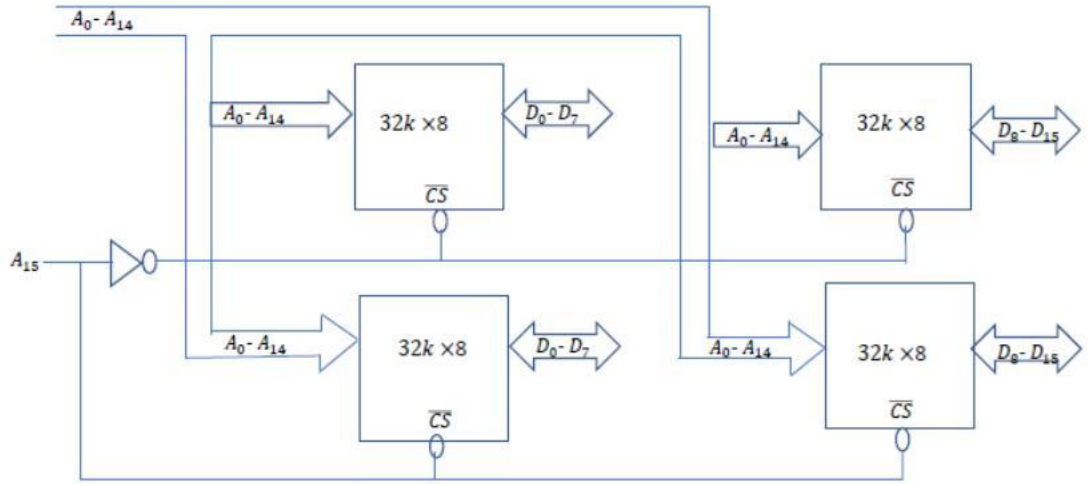
Figure (9)

Arithmetic Microprocessor

Example 2: Implementing 64Kx16 EPROM using 32Kx8 EPROMs

To implement a 64K × 16 EPROM using 32K × 8 EPROMs:

- A 32K × 8 bit EPROM has 15 address lines (A0–A14).



Fig

- A 64K × 16 bit EPROM requires 16 address lines (A0–A15) (2^{16}=64K).
- You would need **four** 32K × 8 EPROMs. Two chips form a 32K × 16 bank, and the A15 address line is used to select between two such 32K × 16 banks, effectively creating a 64K × 16 memory.

82C55 Programmable Peripheral Interface (PPI)

The **82C55 Programmable Peripheral Interface (PPI)** is a versatile interfacing component that connects I/O devices to a microprocessor. commonly found in PCs, often as part of an integrated chipset, for interfacing with the keyboard and parallel printer port

Group A

Port A (PA7-PA0) and upper half of port C (PC7 - PC4)

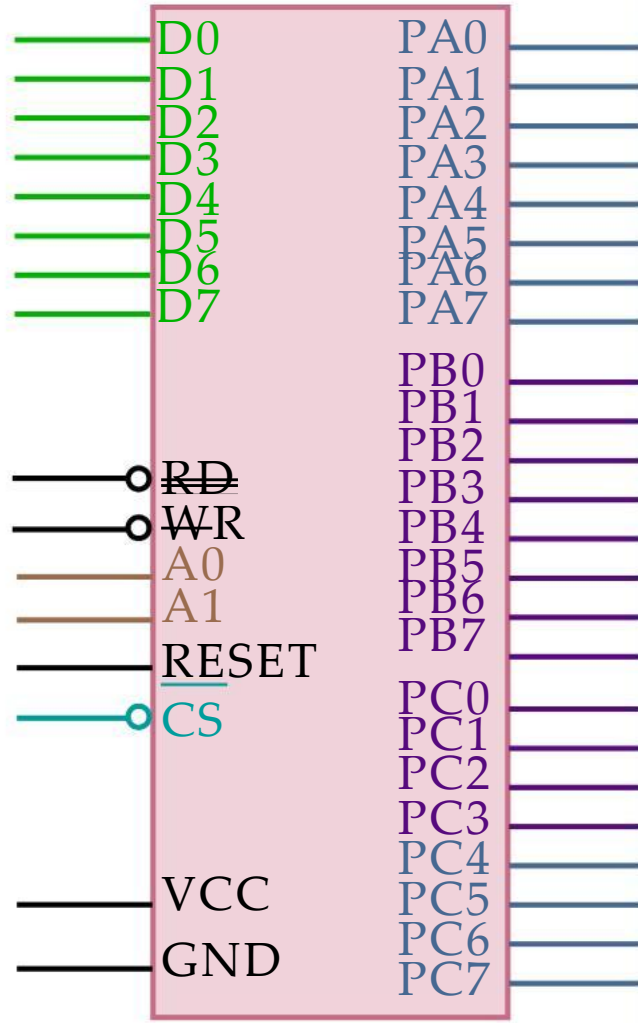
Group B

Port B (PB7-PB0) and lower half of port C (PC3 - PC0)

I/O Port Assignments

A_1	A_0	*Function*
0	0	Port A
0	1	Port B
1	0	Port C
1	1	Command Register

Pinout of 82C55 PPI



Operation Modes of 82C55

The 82C55 PPI operates in:

- **Mode 0 (Simple I/O)**: Provides basic, unhandshaked input/output for general-purpose parallel data transfer.
- **Mode 1 (Strobed I/O)**: Enables handshaked input/output for synchronized data transfer, using Port C for control signals.
- **Mode 2 (Bidirectional Strobed I/O)**: Configures Port A for bidirectional data flow with handshaking, suitable for bused communication.

Mode / Port	Port A (PA0-PA7)	Port B (PB0-PB7)	Port C (PC0-PC7)
Mode 0	8-bit Input or Output (simple I/O)	8-bit Input or Output (simple I/O)	Divided into two 4-bit nibbles, each Input or Output.
Mode 1	8-bit Strobed Input or Output (with handshaking)	8-bit Strobed Input or Output (with handshaking)	Provides handshaking signals for Port A & B.
Mode 2	8-bit Bidirectional Strobed I/O (Port A only)	Cannot be used as Mode 2	Provides handshaking & interrupt control for Port A.

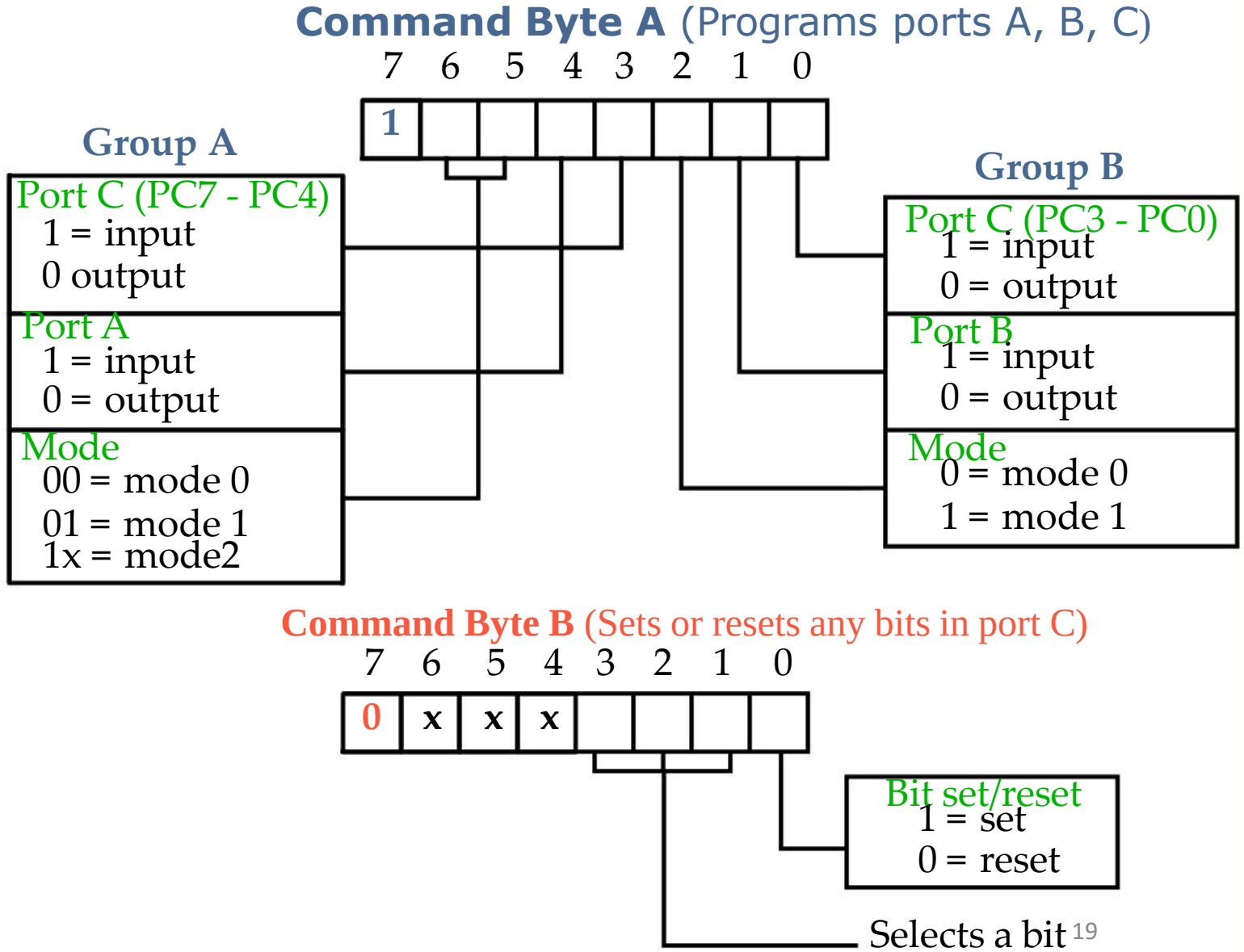
Programming the 82C55

The command byte configures ports A, B, and C as **input or output** and selects Mode 0, 1, or 2.

It controls both **Group A (Port A + upper Port C)** and **Group B (Port B + lower Port C)**.

BSR mode is used to **set or reset individual bits** of **Port C only**, without affecting other ports.

It is selected when bit 7 of the command byte is **0**.



Handshaking

Handshaking is a technique used to **coordinate** data transfer between the CPU and slower I/O devices.It uses **control signals** to confirm that both the sender and receiver are ready before data is sent or received.This avoids data corruption, especially when devices operate at different speeds

Input Handshaking Example –Keyboard

When you press a key, the **keyboard** sends data to the 82C55 and signals it's ready.The 82C55 stores the key data and notifies the **CPU**, which then reads it safely.

Output Handshaking Example –Printer

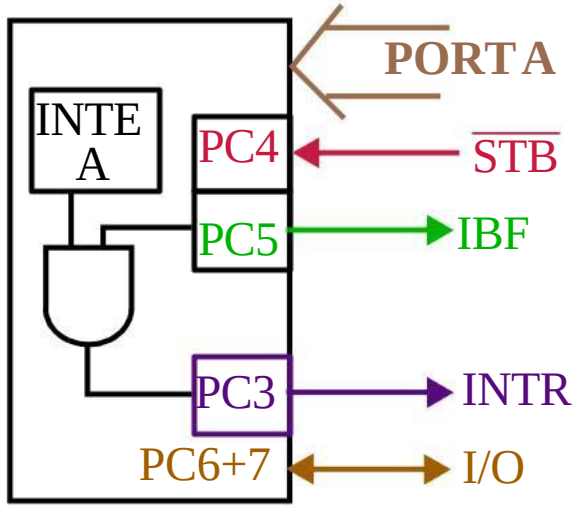
The **CPU** sends a character to the **printer** via 82C55.

The printer sends an **ACK** after receiving it, telling the CPU it's ready for the next one.

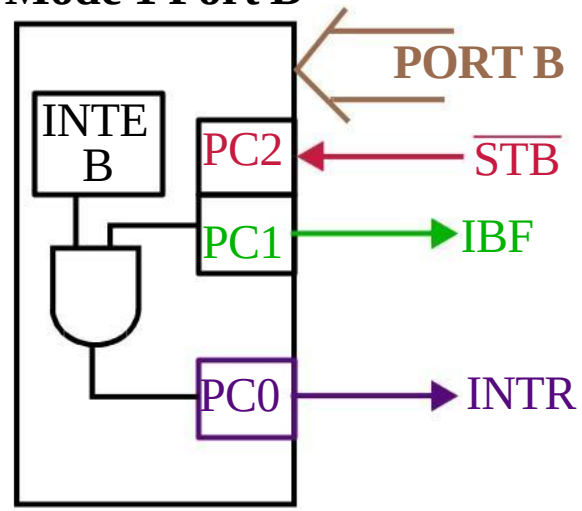
•Handshaking Input (Mode 1 Strobe Input)

82C55: Mode 1 Strobed Input

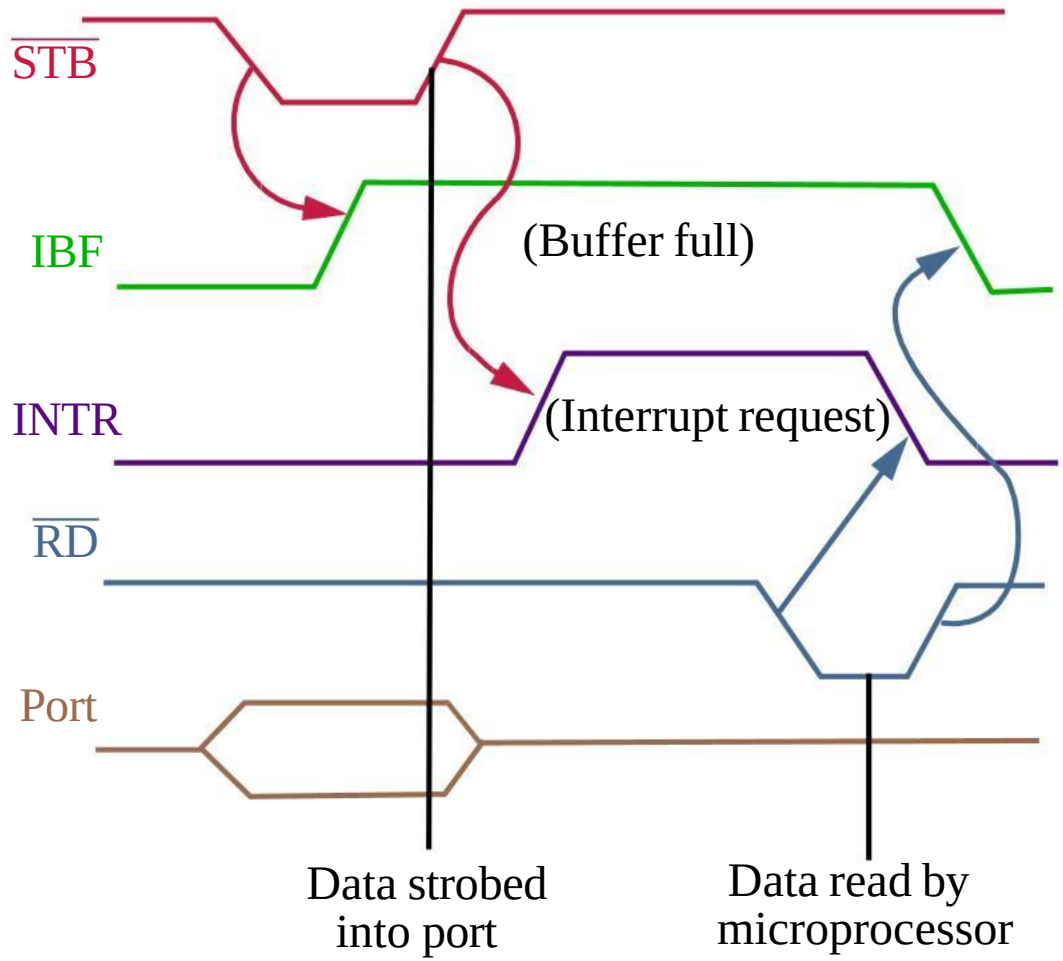
Mode 1 Port A



Mode 1 Port B



Timing Diagram



•Handshaking Input (Mode 1 Strobe Input)

1.STB (Strobe):An external device pulls **STB** low →this tells the 82C55 to latch the incoming data.Data must be valid before STB goes low and held until just after it returns high.

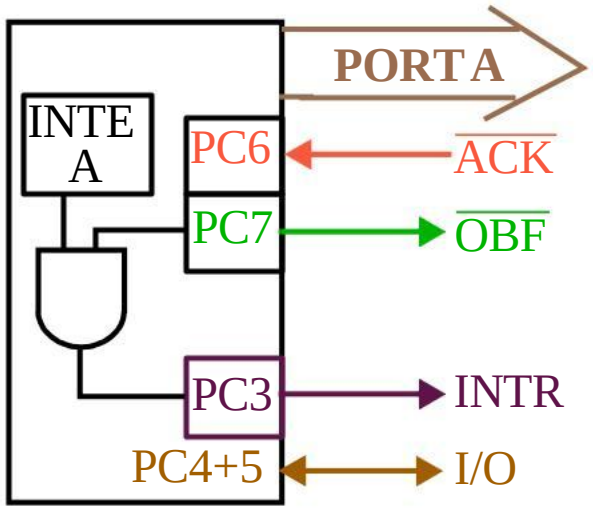
2.IBF (Input Buffer Full):After STB rises, **IBF** goes high to indicate that **valid data is latched** in the input port.It stays high until the CPU reads the data.

3.INTR (Interrupt Request):If interrupts are enabled (via **INTE**), the rising edge of IBF causes **INTR** to go high.This signals the CPU that input data is ready to be read.

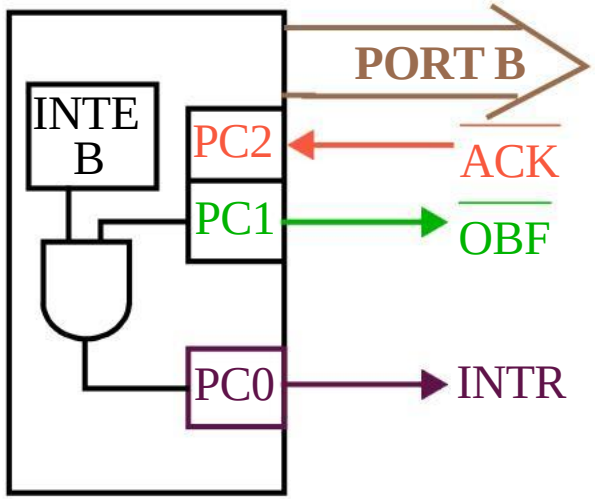
4.RD (Read):The CPU responds by pulling **RD** low to read the data from the port. After the read, IBF and INTR go low, completing the cycle.

•Handshaking Output (Mode 1 Strobe output)

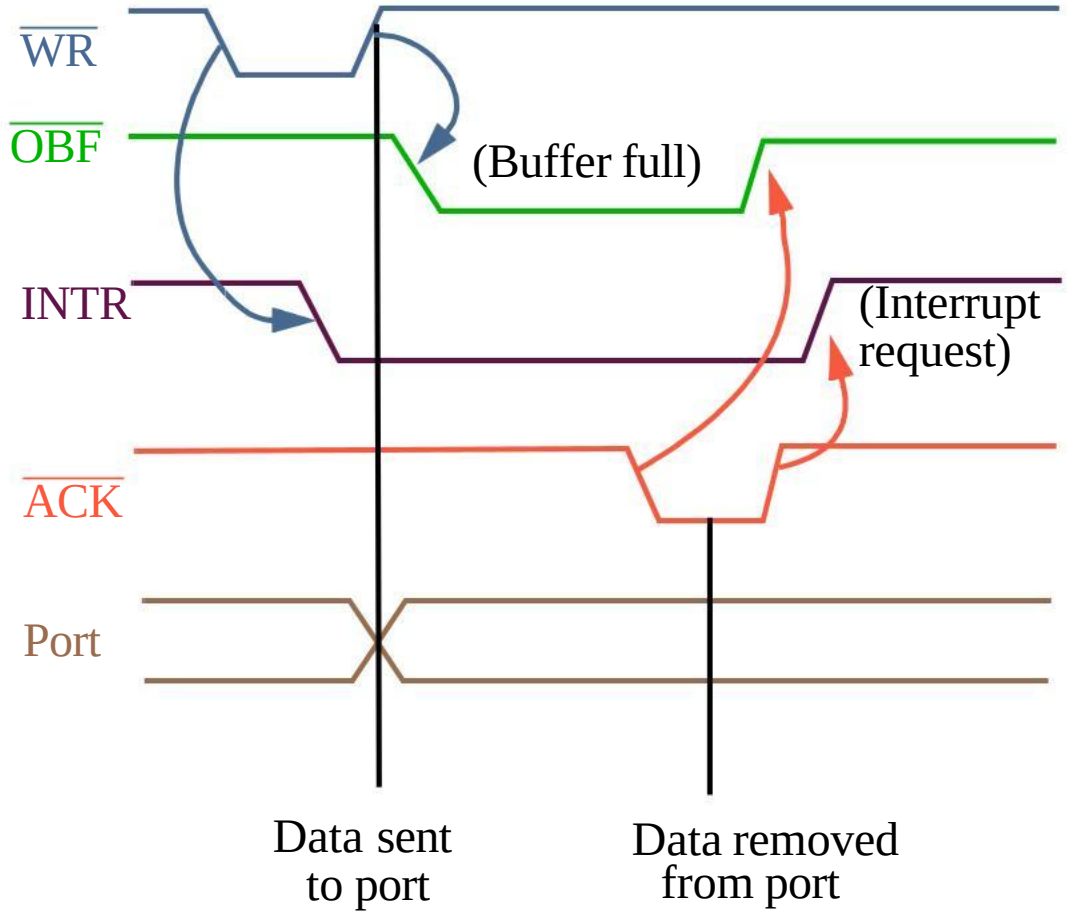
Mode 1 Port A



Mode 1 Port B



Timing Diagram



•Handshaking Output (Mode 1 Strobe output)

1.WR (Write):The CPU places valid data on the bus and pulls **WR** low to latch data into the output port.

2.OBF (Output Buffer Full):After WR goes high, **OBF** goes low to indicate that the port holds **valid output data**.It remains low until the external device acknowledges the data.

3.ACK (Acknowledge):When the external device receives the data, it pulls **ACK** low.This tells the 82C55 that the data was accepted.

4.INTR (Interrupt Request):If **interrupts are enabled** (via INTE), when ACK occurs and OBF goes high again, **INTR** is triggered.

This notifies the CPU that the device is ready for the next data.